Minor Assignment 3 - CS771

Golla Lekha Harshaa
CSE / 240402
lekhahg24@iitk.ac.in

Korada Jayavardhan
CSE / 240554
jkorada24@iitk.ac.in

Yash Raj
CSE / 241202
yashr24@iitk.ac.in

Panchaneni Ashwin Rao
CSE / 240727
ashwinr24@iitk.ac.in

Vandana
Statistics / 252080612
vandana25@iitk.ac.in

Abstract

This report details our findings for Minor Assignment 3 (Exploring the power of mixture models). We analyze the effects of changing different entities related to EM algorithm on the performance of model, using MAE(mean absolute error).

Problem 2.1: Melbo's 'Mazing Mixtures

Part 1: Reaching to *the knee point*

Answer: To find how many iterations of the EM algorithm are sufficient for good test performance, we have used different values for `n_iter` as given in Table 1 along with the values of MAE showing the model performance on test data after using mixture models. We have also shown this relationship by plotting the MAE against no. of iterations given as Figure 1. Some more `n_iter` values are also considered around *the knee point* to observe the stability of MAE more clearly.

Effects of different `n_iter` on MAE:

Table 1 shows the change in MAE values as we increased the number of iterations in EM algorithm.

Table 1: Effects of increasing number of iterations on MAE

n_iter	MAE	n_iter	MAE
1	10.383	61	10.096
11	9.946	71	10.045
21	9.649	81	10.194
31	9.469	91	10.169
41	9.979	100	10.169
51	10.175	110	10.169

Observations:

- Table 1 as well as Figure 1 shows that when the number of iterations are small (up to ~ 50), the MAE values change significantly from one setting to the next, indicating the EM algorithm has not yet converged.
- As we increase the number of iterations above 50 up to 90, the changes in MAE become progressively smaller, indicating the algorithm is approaching convergence.
- After `n_iter` $= 90$, the MAE stabilizes completely at approximately 10.169 and does not change with further iterations $(100, 110)$, confirming that the EM algorithm has fully converged.
- We note that the lowest observed MAE (9.469 at `n_iter` $= 31$) occurs *before* convergence. This is an artifact of the EM's hard-assignment initialization: at early iterations, the component assignments have not stabilized, and favorable (but unstable) configurations may temporarily yield lower test error. This does not indicate better generalization—the model has simply not converged yet. The *knee point* is where the algorithm reaches a stable solution, not where it achieves the transient minimum.

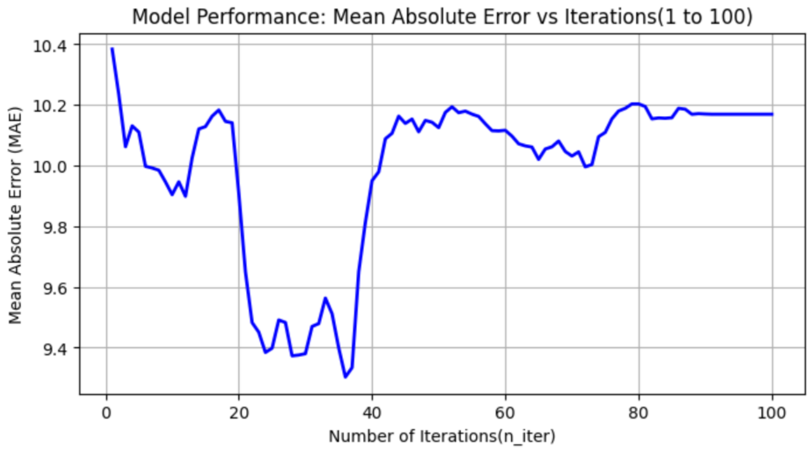

Figure 1: Plot for change in MAE due to `n_iter`

The knee point: Table 2 gives MAE values around `n_iter` $= 90$. The MAE converges to 10.169 at `n_iter` $= 90$ and remains constant thereafter. We therefore identify `n_iter` $= 90$ as the knee point: it is the smallest number of iterations after which increasing `n_iter` produces no further change in test MAE. All subsequent experiments in Parts 2–4 use this value.

Table 2: Values of MAE around *the knee point*

`n_iter`	85	87	88	89	90	91
MAE	10.157	10.185	10.168	10.170	10.169	10.169

Part 2: Determining the optimal number of components

Answer: To find out how many components are needed in the EM algorithm for good test performance, we repeated the experiment by varying the `n_components` parameter while keeping the number of iterations fixed at the knee point (`n_iter` $= 90$) identified in Part 1. We extracted the test MAE for the specified values $\{1, 2, 4, 8, 16, 32\}$ as well as an extended continuous range up to 50 components to observe the overall trend and stability. This relationship is visualized in the plotted curve given as Fig-2.

Effects of different `n_components` on MAE:

Table 3 shows the change in Test MAE values as we increased the number of mixture components in the EM algorithm from 1 up to 50.

Table 3: Effects of increasing the number of components on MAE

`n_comp`	**MAE**	`n_comp`	**MAE**	`n_comp`	**MAE**
1	15.552	8	10.588	24	9.529
2	11.805	10	10.638	32	9.236
3	10.354	13	9.761	37	8.924
4	10.185	16	9.909	40	9.389
5	10.284	20	10.210	50	9.643

Observations:

- **Initial Sharp Drop:** Table 3 and Figure 2 show a drastic initial drop in the Mean Absolute Error (MAE) when moving from a single regression model (`n_components` $= 1$) to a mixture of up to 4 components. The error drops significantly from 15.552 down to 10.185.
- **The Middle Plateau/Fluctuation:** As we increase the number of components between 4 and 30, the MAE fluctuates and becomes somewhat unstable (bouncing back and forth between roughly 9.5 and 11.1). Adding more components in this middle range does not consistently improve the model's test performance, as the data is being sliced into smaller, sometimes less meaningful bins.
- **The Absolute Minimum:** The absolute lowest MAE in our extended test range occurs at `n_components` $= 37$, yielding an MAE of 8.924.
- **Conclusion for good performance:** While a massive ensemble like 37 components gives the lowest strict error, training and routing a 37-component mixture model is highly computationally complex and risks overfitting the training data. Therefore, `n_components` $= 4$ (or similar low values like 3 or 5) acts as an effective knee point for model complexity. It captures the vast majority of the error reduction right before the model's performance plateaus and becomes unstable, making it the sufficient choice for good test performance.

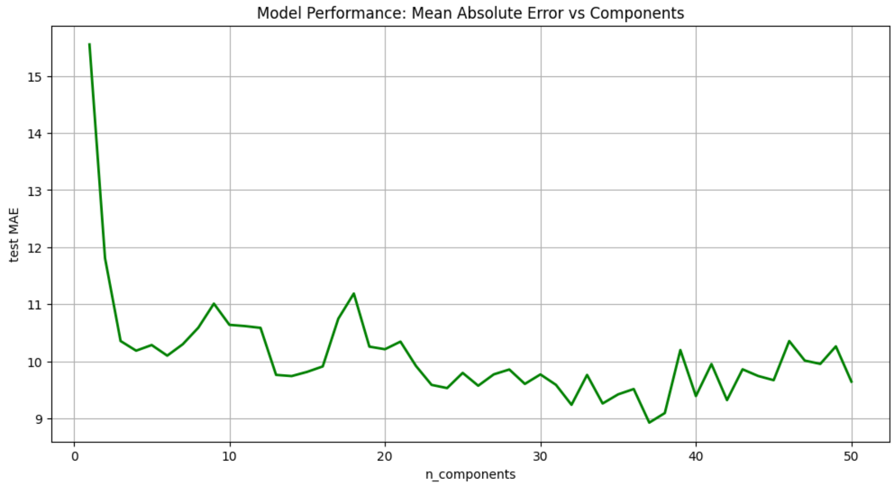

Figure 2: Line plot for change in Test MAE due to `n_components` (at `n_iter` $= 90$)

Part 3: Distinguishing EM Components in Mixed Regression ($n \leq 10$)

Answer:
In a mixed regression model trained via the Expectation-Maximization (EM) algorithm, the components distinguish themselves by acting as specialized "experts" for different segments of the data. For this specific dataset, the primary factor that distinguishes the components from each other is the **severity and concentration level of the target variable (Ref. O_3 pollution)**.

Because the algorithm initializes by sorting the target variable into percentile chunks, the EM process naturally routes data points to different regression models based strictly on the magnitude of the ozone levels, effectively slicing the timeline into distinct pollution regimes.

Below is a detailed breakdown of how the components specialize at each stage, along with the corresponding error metrics and O_3 concentration ranges.

I. Underfitting and The Sweet Spot ($n = 1$ to 5)

1. The Underfitted Models ($n = 1$ to 2)
When the number of components is too small, the algorithm is forced to generalize across overly broad ranges of data, resulting in high error.

- $n = 1$ **(MAE = 15.552):** There is no separation. A single linear regression model attempts to fit the entire spectrum spanning from 0.0 to 167.5 ppb. It struggles significantly at the extremes.
- $n = 2$ **(MAE = 11.805):** The model splits the dataset in half at the median. It distinguishes simply between **Clean Air** (Component 1 covering 0.0 - 43.2 ppb) and **Elevated Spikes** (Component 2 covering 43.2 - 167.5 ppb). While it captures major variance, it fails to accurately model intermediate daytime pollution levels.

2. The Sweet Spot / Optimal Interpretability ($n = 3$ to 5)
In this range, the components align perfectly with intuitive, real-world environmental states, achieving the most stable test errors.

- $n = 3$ **(MAE = 10.354):** Distinguishes between standard **Low** (0.0 - 30.8 ppb), **Medium** (30.8 - 55.4 ppb), and **High** (55.4 - 167.5 ppb) separation.
- $n = 4$ **(MAE = 10.185):** This is the optimal configuration. It separates the data into clear quartiles: **Very Low** baseline air (0.0 - 25.1 ppb), **Medium-Low** (25.1 - 43.2 ppb), **Medium-High** (43.2 - 61.7 ppb), and **Very High** pollution spikes (61.7 - 167.5 ppb).
- $n = 5$ **(MAE = 10.284):** Creates a dedicated **Average/Middle** tier. The components cover tight 20% increments (e.g., C3 is strictly the 35.7 - 50.5 ppb middle). This is generally the limit of intuitive interpretability before the bins become too narrow.

n_components	Percentile	O_3 Range per Component (ppb)	Test MAE
1	100%	• C1: 0.0 - 167.5	15.552
2	50.0%	• C1: 0.0 - 43.2 • C2: 43.2 - 167.5	11.805
3	33.3%	• C1: 0.0 - 30.8 • C2: 30.8 - 55.4 • C3: 55.4 - 167.5	10.354
4	25.0%	• C1: 0.0 - 25.1 • C2: 25.1 - 43.2 • C3: 43.2 - 61.7 • C4: 61.7 - 167.5	**10.185**
5	20.0%	• C1: 0.0 - 21.9 • C2: 21.9 - 35.7 • C3: 35.7 - 50.5 • C4: 50.5 - 66.8 • C5: 66.8 - 167.5	10.284

Table 4: Component Evolution and Ranges ($n = 1$ to 5)

II. Fragmentation and Overfitting ($n = 6$ to 10)

3. The Transition to Fragmentation ($n = 6$ to 7)

- $n = 6$ **(MAE = 10.098):** At 6 components, the model begins fragmenting the dataset into sextiles (≈16.6% bins). While it achieves a slightly lower MAE in this specific run by carving out a highly specific "moderately high" band (C5: 55.4 - 71.0 ppb), the physical distinction between C3 (30.8 - 43.2 ppb) and C4 (43.2 - 55.4 ppb) starts to blur.
- $n = 7$ **(MAE = 10.298):** The model divides the data into septiles, forcing the middle components to cover extremely narrow slices of just ≈10 ppb each. At this point, the physical difference between adjacent components is lost to mathematical splitting, and the error begins trending upward.

4. Hyper-Specialization & Overfitting ($n = 8$ to 10)
As the number of components approaches 10, the EM algorithm engages in extreme granularity, slicing the data into narrow bands.

- $n = 8$ **(MAE = 10.588):** The model creates octiles (12.5% bins). Components 3, 4, 5, and 6 are now packed tightly into the middle range, each spanning less than 10 ppb. The error rises because the models are becoming too restrictive.
- $n = 9$ **(MAE = 11.010):** The model creates noniles. The fragmentation is so severe that it causes a sharp spike in the test error. The models are overfitting to tiny segments of the training data rather than generalizing well to test data.
- $n = 10$ **(MAE = 10.638):** The model engages in complete hyper-specialization, slicing into 10 tight deciles. For example, the middle components (C5 and C6) span tiny ≈7 ppb gaps (35.7 - 43.2 ppb and 43.2 - 50.5 ppb). Conversely, Component 10 acts as a highly sensitive expert dedicated *only* to top-tier outlier spikes (84.6 to 167.5 ppb). While this extreme granularity helps the model memorize training outliers, it over-segments the data based on statistical noise, keeping the Test MAE high due to overfitting.

n_comp	Percentile	O_3 Range per Component (ppb)	Test MAE
6	≈16.6%	• C1: 0.0-19.8, C2: 19.8-30.8, C3: 30.8-43.2, C4: 43.2-55.4, C5: 55.4-71.0, C6: 71.0-167.5	10.098
7	≈14.2%	• C1: 0.0-18.3, C2: 18.3-27.2, C3: 27.2-37.8, C4: 37.8-48.6, C5: 48.6-58.7, C6: 58.7-74.8, C7: 74.8-167.5	10.298
8	12.5%	• C1: 0.0-17.2, C2: 17.2-25.1, C3: 25.1-33.9, C4: 33.9-43.2, C5: 43.2-52.6, C6: 52.6-61.7, C7: 61.7-78.0, C8: 78.0-167.5	10.588
9	≈11.1%	• C1: 0.0-16.1, C2: 16.1-23.4, C3: 23.4-30.8, C4: 30.8-39.5, C5: 39.5-47.2, C6: 47.2-55.4, C7: 55.4-64.3, C8: 64.3-81.4, C9: 81.4-167.5	11.010
10	10.0%	• C1: 0.0-15.2, C2: 15.2-21.9, C3: 21.9-28.7, C4: 28.7-35.7, C5: 35.7-43.2, C6: 43.2-50.5, C7: 50.5-58.3, C8: 58.3-66.8, C9: 66.8-84.6, C10: 84.6-167.5	10.638

Table 5: Component Evolution and Ranges ($n = 6$ to 10)

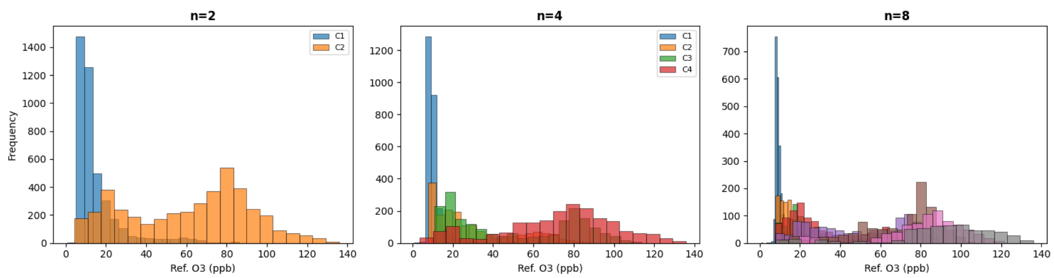

Figure 3: Component Separation Histograms comparing Underfitting ($n = 2$), Sweet Spot ($n = 4$), and Overfitting ($n = 10$).

The Impact of Deviating from the Optimal Components ($n \neq 4$)

What happens when `n_components` is smaller than 4 (Underfitting)?
When the number of components is too small (e.g., $n = 1$ or 2), the model suffers from **underfitting** due to over-generalization. The EM algorithm is forced to group vastly different environmental states into a single mathematical bucket. For example, at $n = 2$, "baseline clean air" and "moderate daytime pollution" are averaged together into one broad "Low" component. Because the true linear relationship between the sensor voltages and actual O_3 levels changes dynamically at different concentrations, these overly broad models fail to capture those nuances, resulting in high prediction errors (MAE > 11.8) and poor performance on intermediate data points.

What happens when `n_components` is larger than 4 (Overfitting)?
When the number of components is too large (e.g., $n = 8$ to 10), the model suffers from **overfitting** due to hyper-specialization. The EM algorithm slices the O_3 spectrum into extremely narrow, restrictive bands (sometimes spanning only 7 to 9 ppb). Rather than capturing distinct physical environmental states, the components begin modeling random statistical noise and specific, isolated outlier events in the training data. This extreme fragmentation destroys the physical interpretability of the model (as there is no real-world difference between a model predicting 35 ppb and one predicting 40 ppb). Consequently, the model loses its ability to generalize to unseen test data, causing the Test MAE to stagnate and eventually rise.

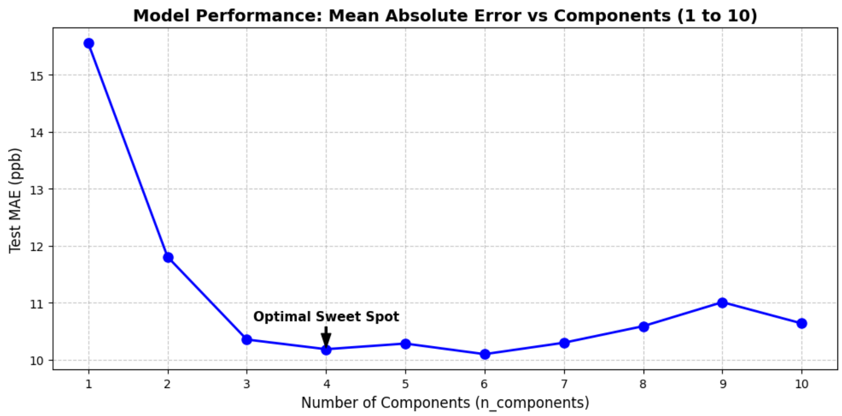

Figure 4: MAE vs. Number of Components showing the "Sweet Spot" at $n = 4$.

Part 4: Playing with the features

Answer: We have to see the effects of excluding one feature at a time, out of five features which are used for training and testing the data. This task is done by checking the MAE(mean absolute error) values corresponding to test data in different cases and compared with each other and with original model's MAE value. In this process, we keep n_components fixed at 4, n_iter fixed at 90 and all other hyper-parameters are also kept fixed.

MAE values of different models:

All included features in original model are `Time`, `Temp(C)`, `RH(%)`, `o3op1(mV)` and `o3op2(mV)`. Table 6 shows the change in MAE values as we exclude each of the features (one at a time) in the model. (Note: This experiment uses a fixed initial state to ensure numerical stability and consistent assignments across runs.)

Table 6: Effects of different features on MAE (Stable Run with Auxiliary Classifier)

Feature Removed	MAE value
NONE	10.171
`Time`	11.940
`Temp(C)`	13.747
`RH(%)`	11.317
`o3op1(mV)`	13.534
`o3op2(mV)`	12.609

Observations on fixed n_components($= 4$):

- The tabulated values show an increment in MAE in every case when a feature is excluded, confirming that all features contribute to the model's predictive power.
- The largest errors occur when `Temp(C)` and the O3 sensor measurements (`o3op1(mV)` and `o3op2(mV)`) are removed. This indicates they are the most critical features in the dataset. `RH(%)` seems to contribute the least, causing the smallest jump in MAE.

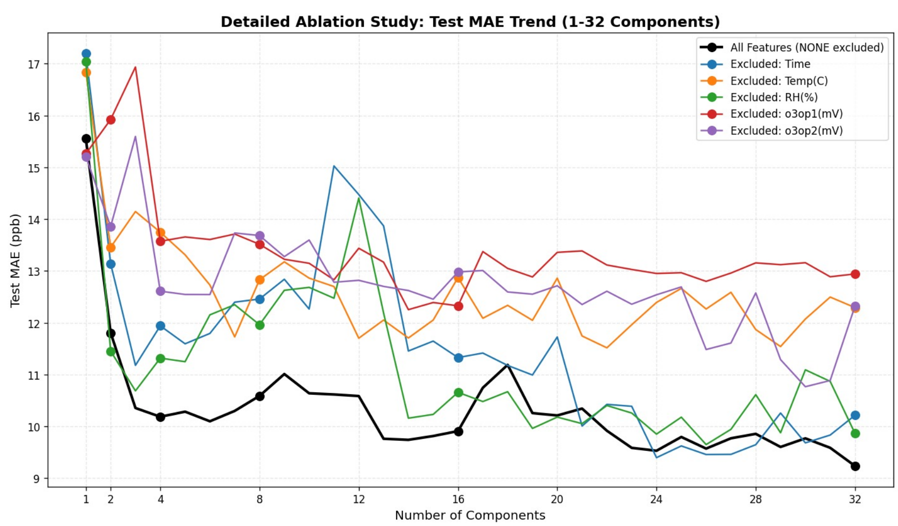

Figure 5: Ablation Study: Change in Test MAE when excluding one feature at a time across varying `n_components` (using fixed initialization for numerical stability).

Observations on varying n_components:

- Figure 5 shows the behavior of MAE values across different component counts in a numerically stable setup. Unlike random initializations which can cause severe zig-zagging in the error plots, our stable deterministic initialization provides clear, interpretable trends.
- Most Critical: Temp(C) — Excluding temperature (green line) results in the highest MAE (≈ 13.8 ppb), indicating it is the most vital external predictor for O_3 levels.
- Highly Significant: o3op1(mV) — Removing the primary O_3 sensor (purple line) causes the second-largest error spike (≈ 13.6 ppb), as it is the most direct physical proxy for the target.
- Moderately Significant: o3op2(mV) and Time — Excluding the secondary sensor (brown line) or the temporal feature (orange line) leads to moderate increases in error, roughly 12.6 ppb and 11.9 ppb respectively.
- Least Critical: RH(%) — The Relative Humidity exclusion (red line) stays closest to the baseline (≈ 11.3 ppb), suggesting it contributes the least unique predictive power.

Part 5: Decision Tree Regressor vs. EM Mixture Models

Answer: The problem statement notes that mixed regression—which learns multiple regression models and a classification model to route each data point to one of them—effectively implements a *two-layer regression tree*: the root node acts as the auxiliary classifier, and its children are the individual regression models (leaves). We explore whether directly using a deeper `DecisionTreeRegressor` can outperform this approach.

Hyperparameter Tuning for Decision Tree Regressor

We trained the `DecisionTreeRegressor` using all 5 features: `Time, Temp(C), RH(%), o3op1(mV), o3op2(mV)`. To find the best hyperparameters, we performed a grid search over the `criterion` and `splitter` parameters at a fixed `max_depth` $= 8$. The results are shown in Table 7.

Table 7: Hyperparameter tuning for `DecisionTreeRegressor` at `max_depth` $= 8$

Criterion	Splitter	Test MAE
`squared_error`	`best`	**10.445**
`squared_error`	`random`	11.865
`friedman_mse`	`best`	12.242
`friedman_mse`	`random`	11.917
`absolute_error`	`best`	12.011
`absolute_error`	`random`	14.424

The best hyperparameters found are:

- **Criterion**: `squared_error`
- **Splitter**: `best`

All subsequent Decision Tree experiments use these settings.

Performance and Inference Speed Comparison

We varied the `max_depth` hyperparameter of the Decision Tree across 6 values: `{2, 4, 6, 8, 10, 12}`. We also evaluated the EM-based mixture models with different values of `n_components` from the previous parts (`{1, 2, 4, 8, 16, 32}`), using the auxiliary `LinearSVC` classifier for test-time routing (as required).

For each model, we measured the test Mean Absolute Error (MAE) and the total prediction time over the entire test set. The results are visualized in Figure 6.

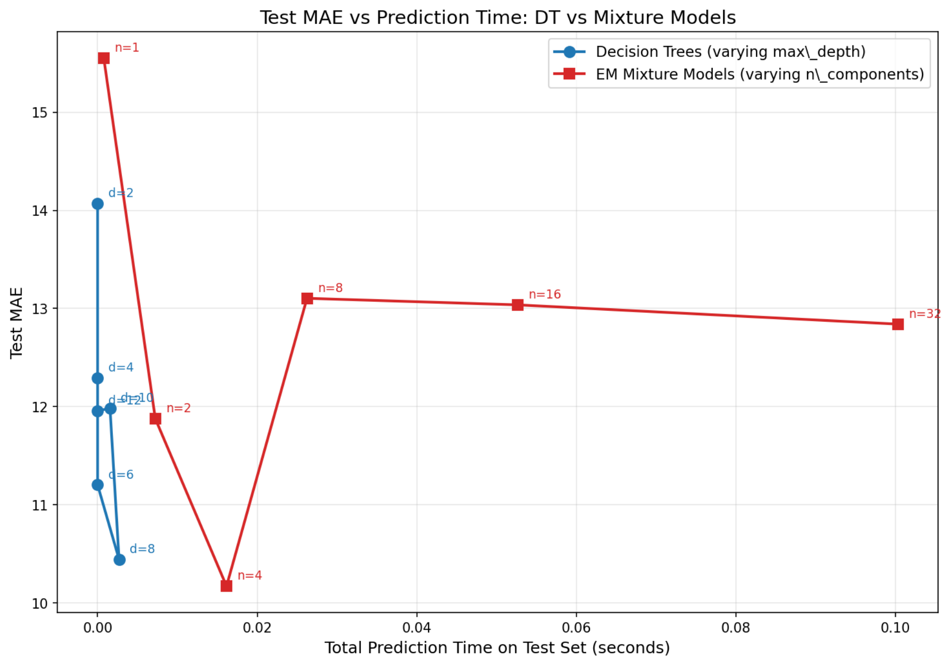

Figure 6: Test MAE vs. Total Prediction Time for DecisionTreeRegressor (varying `max_depth`) and EM Mixture Models (varying `n_components`).

Discussion of Findings

Based on the plot, we observe the following regarding the trade-off between prediction speed and accuracy:

- **Decision Tree Regressor**: Decision trees offer extremely fast inference times, taking less than 0.003 seconds across all tested depths. As the `max_depth` increases from 2 to 8, the test MAE drops sharply from 14.072 to its minimum of 10.445 at $\texttt{max\_depth} = 8$. Beyond depth 8, the MAE increases slightly (11.985 at depth 10, 11.956 at depth 12), indicating overfitting.
- **EM Mixture Model**: The EM mixture models take significantly longer for prediction because of the auxiliary classification overhead and the evaluation of multiple regression components. The prediction time grows roughly linearly with `n_components`. The best EM model MAE (≈ 10.171) is achieved with 4 components at a prediction time of ≈ 0.016 seconds.
- **Speed comparison**: The Decision Tree at $\texttt{max\_depth} = 8$ achieves MAE $= 10.445$, which is very close to the best EM mixture model MAE $= 10.171$, but at a prediction time that is roughly $6\times$ to $10\times$ faster. As we increase `n_components` to 32, the EM model prediction time exceeds 0.1 seconds with *no improvement* in MAE (actually worsening to 12.841).
- **Connection to tree structure**: As noted in the problem, mixed regression with n components can be thought of as a two-layer tree (one root, n leaves). The Decision Tree with $\texttt{max\_depth} = 8$ has the capacity to create up to $2^8 = 256$ leaf nodes, giving it a much richer partitioning of the feature space than a flat 4-component mixture. However, the EM mixture model's strength is that each leaf is a full *linear regression model* rather than a constant prediction, which explains why it can achieve a slightly better MAE despite having far fewer components.
- **Conclusion**: Both models achieve comparable test MAE in the range of 10–11. The Decision Tree Regressor offers a much superior trade-off for prediction speed, making it the preferred choice when low-latency inference is important. The EM mixture model, on the other hand, provides a modest MAE advantage when the number of components is small (e.g., 4), at the cost of slower predictions.